一、模块化的基本概念

模块只是一个代称，它就像是一块砖（内部封装了多段代码），哪里需要就往哪里搬

API 也是模块

1.什么是模块化

- 模块化是指解决一个每杂问题时,**自顶向下**逐层把系统划分成若干模块的过程。对于整个系统来说,**模块是可组合、分解和更换的单元。**
- **编程领域中的模块化**,就是遵守固定的规则,把**一个大文件**拆成**独立并互相依赖的多个小模块。**
- 把代码进行模块化拆分的好处:
    - 提高了代码的**复用性**
    - 提高了代码的**可维护性**
    - 可以实现**按需加载**

2.模块化规范

模块化规范就是对代码进行模块化的拆分与组合时,**需要遵守的那些规则。**

例如:

- 使用什么样的语法格式来**引用模块**
- 在模块中使用什么样的语法格式**向外暴露成员**

模块化规范的好处:大家都遵守同样的模块化规范写代码,降低了沟通的成本,极大方便了各个模块之间的相互调用, 利人利己。

Node.js使用的模块系统主要有两种：CommonJS和ESM（ES Modules）

CommonJS是Node.js最早使用的模块系统,它使用**require和module.exports** 来导入和导出模块。

ESM(ES Modules)是标准化的JavaScript模块系统,它在Node.js 12及更高版本中得到了原生支持。ESM使用**import和export** 语法来导入和导出模块。

二、Node.js中的模块化

1. Node.js 中模块的分类

Node.js 中根据模块来源的不同,将模块分为了3大类,分别是:

- **内置模块**(内置模块是由Node.js官方提供的,例如fs、path、http等)

- **自定义模块**(用户创建的每个.js文件,都是自定义模块)
- **第三方模块**(由第三方开发出来的模块,并非官方提供的内置模块,也不是用户创建的自定义模块,使用前需要先下载)

补充：API分类 此处的模块 即 API

1. **ECMAScript 内置API（语言本身）** Array、Promise、Symbol、Math、JSON 等，V8引擎实现，浏览器、Node都自带。
2. **运行环境原生API（环境自带）** 浏览器：DOM、BOM、fetch、setTimeout Node：fs、path、http、process 由浏览器内核 / Node 的libuv等底层提供，不属于ES规范。
3. **自定义API（自己封装）** 你自己写函数、类、模块，对外暴露方法，就是你自己的API。 比如你封装一个 `getUserInfo()`，其他代码调用它，这就是自定义API。

```
// 自定义API
function getUserInfo(id) {
  return { id, name: "xxx" }
}
export { getUserInfo }
```

4.**第三方API（npm包）** 别人写好发布到npm，你安装后引入使用。 比如 axios、lodash、mongoose、 nestjs 这些。 本质上也是别人写好的**自定义代码封装**，只是打包上传到npm仓库共享。

> 注意：npm包只是JS代码封装，底层最终还是依赖上面两类：ES内置API + 运行环境API（fs、http等）

2.加载模块

使用强大的require()方法,可以加载需要的内置模块、用户自定义模块、第三方模块进行使用。例如:

```
//1. 加载内置的 fs 模块
const fs = require('fs')

// 2. 加载用户的自定义模块 PS：后缀名 .js 可以省略
const custom = require('/custom.js')

// 3. 加载第三方模块(需先下载，然后再使用)
const moment = require('moment')
```

注意:使用require()方法加载其它模块时,**会执行被加载模块中的代码。**若是有打印内容，会直接输出

3.Node.js中的模块作用域

- **模块作用域**：和函数作用域类似,**在自定义模块中定义的变量、方法等成员,只能在当前模块内被访问**,这种模块级别的访问限制,叫做模块作用域。

加载一个自定义模块，之后会得到一个空对象，无法访问被加载模块中的私有成员

- 模块作用域的好处：
    - 防止了全局变量污染的问题

4.向外共享模块作用域中的成员

4.1 module对象

在每个.js自定义模块中都有一个**module对象,它里面存储了和当前模块有关的信息**,打印如下:

```
console.log(module)
```

```
{
  id: '.',
  path: 'D:\\实习\\实习生全栈学习笔记\\Part1_Node.js\\1104\\code',
  exports: {},
  filename: 'D:\\实习\\实习生全栈学习笔记\\Part1_Node.js\\1104\\code\\03.演示module对象.js',
  loaded: false,
  children: [],
  paths: [
    'D:\\实习\\实习生全栈学习笔记\\Part1_Node.js\\1104\\code\\node_modules',
    'D:\\实习\\实习生全栈学习笔记\\Part1_Node.js\\1104\\node_modules',
    'D:\\实习\\实习生全栈学习笔记\\Part1_Node.js\\node_modules',
    'D:\\实习\\实习生全栈学习笔记\\node_modules',
    'D:\\实习\\node_modules',
    'D:\\node_modules'
  ],
  Symbol(kIsMainSymbol): true,
  Symbol(kIsCachedByESMLoader): false,
  Symbol(kURL): undefined,
  Symbol(kFormat): undefined,
  Symbol(kIsExecuting): true
}
```

4.2 module.exports 对象

- 在自定义模块中,可以使用**module.exports对象**,将模块内的成员共享出去,供外界使用。
- **外界用require()方法导入自定义模块时,得到的就是module.exports所指向的对象。**

在一个自定义模块中，默认情况下，module.exports是一个空对象,即module.exports = {}，所以当外界通过require()方法加载该模块时，返回的是一个空对象。

```
// 在一个自定义模块中，默认情况下，module.exports是一个空对象,即module.exports = {}，所以当外界通过require()方法加载该模块时，返回的是一个空对象。

// 向 module.exports 对象上挂载 username 属性
module.exports.username = '张三'

// 向 module.exports 对象上挂载 sayHello 函数
module.exports.sayHello = function () {
  console.log('大家好，我是' + this.username)
}

const age = 18
module.exports.age = age
```

```
// 在外界使用require 导入一个自定义模块的时候，得到的成员
// 就是 那个模块中，通过 module.exports 指向的对象。

const m = require('./04.自定义模块.js')  // 后缀可以省略
console.log(m)
// 输出内容：
// { username: '张三', sayHello: [Function: sayHello], age: 18 }
```

4.3 共享对象时的注意点

使用require()方法导入模块时,导入的结果,**永远以module.exports指向的⭐⭐对象⭐⭐为准。** 相当于一个变量,变量值等于最后一次赋值的值

```
// 向 module.exports 对象上挂载 username 属性
module.exports.username = '张三'

// 向 module.exports 对象上挂载 sayHello 函数
module.exports.sayHello = function () {
  console.log('大家好，我是' + this.username)
}

// 让 module.exports 指向⭐⭐一个全新的对象 ⭐⭐
module.exports = {
    username: '小禹',
    sayHello: function () {
        console.log('大家好，我是' + this.username)
    }
}
```

```
const m = require('./06.自定义模块.js')  // 后缀可以省略
console.log(m)
// 输出内容：
// { username: '小禹', sayHello: [Function: sayHello] }
```

4.4 exports 对象

由于module.exports单词写起来比较复杂,为了简化向外共享成员的代码,Node提供了exports对象。默认情况下,exports 和module.exports指向同一个对象。**最终共享的结果,还是以module.exports指向的对象为准。**

```
console.log(exports)
console.log(module.exports)

console.log(module.exports === exports)

// {}
// {}
// true
```

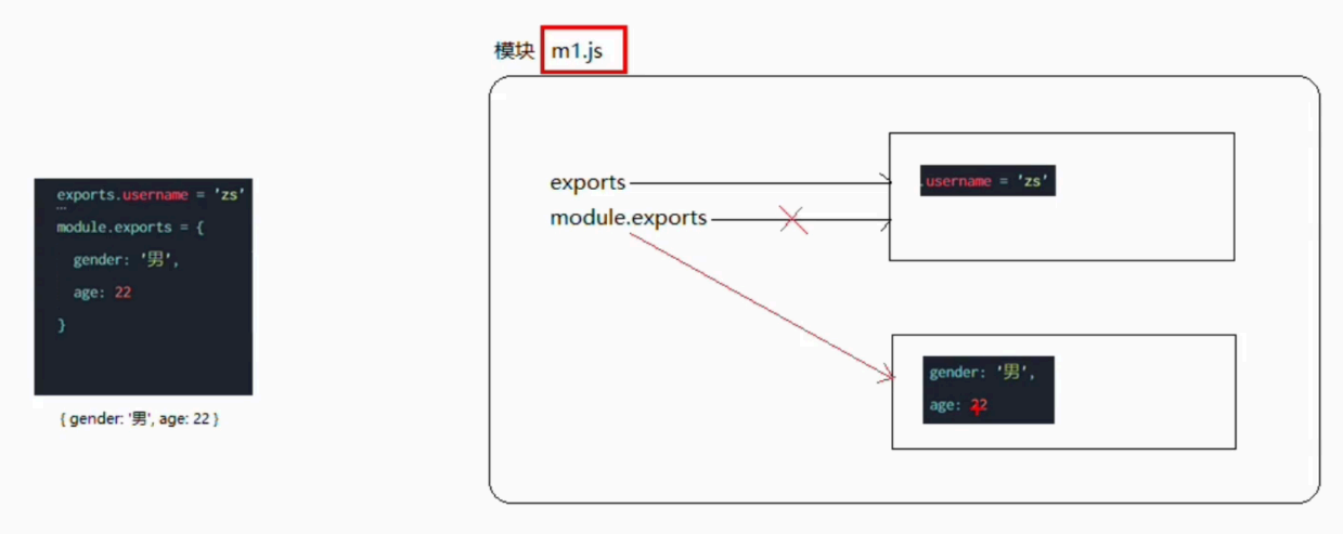
4.5 exports 和 module.exports 的使用误区

注意区分对象和对象中的==属性

时刻谨记,require()模块时,得到的**永远是module.exports指向的对象**:

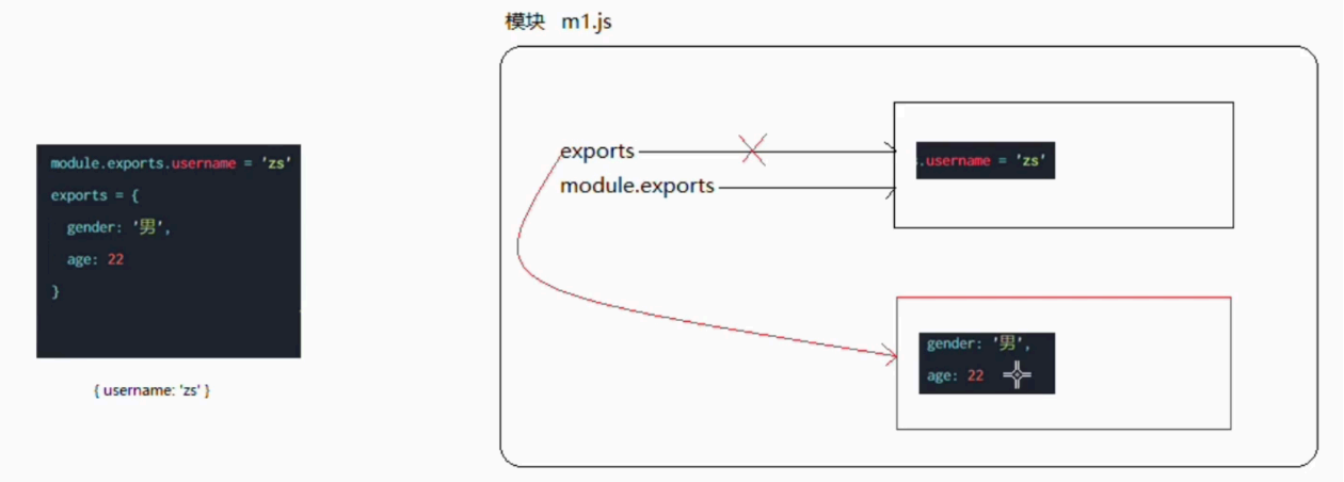
```
exports.uesrname = 'zs'
module.exports = {
    gender:'男',
    age:22
}
```

{gender:'男',age:22}




```
module.exports.username = 'zs'
exports = {
    gender:'男',
    age:22
}
```

{username:'男'}



PS：**注意区分对象和对象中的属性**

```
exports.username = 'zs'
module.exports.gender = '男'
```

{username:'zs',gender='男'}

```
exports = {
    username:'zs',
    gender:'男'
}

module.exports = exports
module.exports.age = 22
```

{username:'zs',gender:'男'，age：22}

为了防止混乱,建议不要在同一个模块中同时使用exports和module.exports

5.Node.js 中的CommonJS模块化规范

CommonJS规定了模块的特性和各模块之间如何相互依赖。

CommonJS 规定:

- 每个模块内部,**module 变量**代表当前模块。
- module变量是一个对象,它的exports属性(即**module.exports**)是对外的接口。
- 加载某个模块,其实是加载该模块的module.exports属性。**require()**方法用于加载模块。

三、npm与包

1.包

- Node.js 中的**第三方模块**又叫做**包。**

就像电脑和计算机指的是相同的东西,第三方模块和包指的是同一个概念,只不过叫法不同。

- 需要包的原因:
  - 由于Node.js的内置模块仅提供了一些底层的API（如fs、path、http),导致在基于内置模块进行项目开发的时侯,效率很低。
  - **包是基于内置模块封装出来的**,提供了更高级、更方便的API,极大的提高了开发效率。
- 地址:
  - https://www.npmjs.com/，全球最大的包共享平台 ——> 搜素所需的包
  - https://registry.npmjs.org/的服务器,来对外共享所有的包——>提供可供下载的包
  - 包管理工具：**Node Package Manage**r(简称npm包管理工具)。使用这个包管理工具,从https://registry.npmjs.org/服务器把需要的包下载到本地使用

    `npm -v` 查看npm的版本号

2.npm与包

2.1 格式化时间的传统做法

1 创建格式化时间的自定义模块

2 定义格式化时间的方法

3 创建补零函数

4 从自定义模块中导出格式化时间的函数

5 导入格式化时间的自定义模块

6 调用格式化时间的函数

```
// 定义格式化时间的方法
function dateFormat (daStr) {
    const dt = new Date(daStr)

    const y = dt.getFullYear()
    const m = padZero(dt.getMonth() + 1)
    const d = padZero(dt.getDate())

    const h = dt.getHours()
    const mm = padZero(dt.getMinutes())
```

```
    const ss = padZero(dt.getSeconds())
    return `${y}-${m}-${d} ${h}:${mm}:${ss}`
}

// 定义补零的方法
function padZero (n) {
    return n < 10 ? '0' + n : n
}

module.exports = {
    dateFormat
}
```

```
// 导入自定义的格式化时间的模块
const TIME = require('./09.dateFormat.js')

// 调用格式化时间的方法
// console.log(TIME.dateFormat(new Date()))
const dt = new Date()
const newDT = TIME.dateFormat(dt)
console.log(newDT)

// 输出内容:
// 2023-11-04 10:20:30
```

2.2 格式化时间的高级做法

1 使用npm包管理工具,在项目中安装格式化时间的包moment

2 使用require()导入格式化时间的包,

3 参考moment的官方API文档对时间进行格式化

```
//1.导入 moment 包
const moment = require( 'moment')

// 2. 参考 moment 官方 API 文档,调用对应的方法,对时间进行格式化
// 2.1 调用 moment()方法,得到当前的时间
// 2.2 针对当前的时间,调用format()方法,按照指定的格式进行时间的格式化
const dt = moment(). format( 'YYYY-MM-DD HH:mm: ss')

console.log(dt)//输出 2020-01-12 17:23:48
```

2.3 在项目中安装包的命令

```
npm install 完整包名称
npm uninstall  完整的包名称
npm uni 完整的包名称

npm i    读取package.json，一次性安装所有所需的包
npm search 包名
```

简写格式：

```
npm i 完整的包名称  -->默认下载最新的包    npm i xx xx xx
npm i 包名称@版本号   -->下载指定版本的包
```

- 在某一目录下初次执行了 `npm i xxx` 过后，该目录下会自动生成：`packagelock.json`，`package.json`，`node_modules/`
    - **node_modules文件夹**用来存放所有已安装到项目中的包。require()导入第三方包时,就是从这个目录中查找并加载包。
    - **package-lock.json** 配置文件用来记录node_modules目录下的每一个包的下载信息,例如包的名字、版本号、下载地址等。
    - 在package.json 文件中，有一个 **dependencies 节点**，专门用来记录使用npm install 安装了哪些包
    - 注意:不要手动修改node_modules或package-lock.json文件中的任何代码,npm包管理工具会自动维护它们。

  也可以在执行命令所处的目录中，执行`npm init -y`，建立 package.json文件（PS:该命令只能在英文的目录下成功运行）
- 在https://www.npmjs.com/中搜索包，然后再document中查看这个包的用法
- 包的版本号： 大版本.功能版本.Bug修复版本
    - 版本号提升的规则:只要前面的版本号增长了,则后面的版本号归零。
- `npm install (或 npm i)`一次性安装所有的包
    - 执行 npm install 命令时,npm 包管理工具会先读取 package.json 中的 dependencies 节点,读取到记录的所有依赖包名称和版本号之后,npm 包管理工具会把这些包一次性下载到项目

补充：

- **devDependencies 节点**
    - 如果某些包**只在项目开发阶段会用到**,在项目上线之后不会用到,则建议把这些包记录到deyDependencies节点中。
    - 与之对应的,如果某些包在开发和项目上线之后都需要用到,则建议把这些包记录到dependencies节点中。

```
//安装指定的包,并记录到 devDependencies 节点中
npm i 包名 -D
// 注意:上述命令是简写形式,等价于下面完整的写法:
npm install 包名 -- save-dev
```

- nrm

2.4 包的分类

使用npm包管理工具下载的包,共分为两大类,分别是:

- **项目包**

  那些被安装到项目的node_modules 目录中的包,都是项目包。又可以分为两类:

  - **开发依赖包**(被记录到devDependencies节点中的包,只在开发期间会用到)
  - **核心依赖包**(被记录到dependencies节点中的包,在开发期间和项目上线之后都会用到)

```
npm i 包名 -D   # 开发依赖包(会被记录到devDependencies 节点下)
npm i 包名      # 核心依赖包(会被记录到 dependencies 节点下)
```

- **全局包**

  在执行 npm install命令时,如果提供-g参数,则会把包安装从全局包。

  全局包会被安装到C:\Users\用户目录\AppData\Roaming\npm\node_modules目录下。

```
npm i 包名 -g             # 全局安装指定的包
npm uninstall 包名 -g     #卸载全局安装的包
```

  只有工具性质的包,才有全局安装的必要性。因为它们提供了好用的终端命令。 **判断某个包是否需要全局安装后才能使用,可以参考官方提供的使用说明即可**（https://www.npmjs.com/）。

补充:

- **i5ting_toc是一个可以把md文档转为html页面的小工具**,使用步骤如下:

```
npm install -g i5ting_toc  # 安装为全局包

#调用 i5ting_toc,轻松实现 md 转 html 的功能
i5ting_toc -f 要转换的md文件路径 -o      # -o 这个参数表示转换完成后，直接在浏览器展示
```

四、模块的加载机制

1.优先从缓存中加载

模块在第一次加载后会被缓存。这也意味着多次调用require()不会导致模块的代码被执行多次。

注意:不论是内置模块、用户自定义模块、还是第三方模块,它们都会**优先从缓存中加载,从而提高模块的加载效率。**

```
console.log('hello ')
```

```
require('./12.验证模块的缓存加载机制.js')
require('./12.验证模块的缓存加载机制.js')
require('./12.验证模块的缓存加载机制.js')

// 只会 输出一行 hello
// 模块在第一次加载后会被缓存。这也意味着多次调用require()不会导致模块的代码被执行多次。
```

2.内置模块的加载机制

内置模块是由Node.js官方提供的模块,内置模块的加载优先级最高。

例如,require('fs')始终返回内置的fs模块,即使在node_modules目录下有名字相同的包也叫做fs。

3.自定义模块的加载机制

使用**require()加载自定义模块**时,必须指定**以/或 .. /开头的路径标识符。**在加载自定义模块时,如果没有指定./或 .. / 这样的路径标识符,则node会把它当作内置模块或第三方模块进行加载。

同时,在使用require()导入自定义模块时,如果省略了文件的扩展名,则Node.js会按顺序分别尝试加载以下的文件:

1 按照确切的文件名进行加载

2 补全.js扩展名进行加载

3 补全.json扩展名进行加载

4 补全.node扩展名进行加载

5 加载失败,终端报错

4.第三方模块的加载机制

如果传递给require()的模块标识符不是一个内置模块,也没有以"或"开头,则Node.js会从当前模块的父 目录开始,尝试从/node_modules文件夹中加载第三方模块。

如果没有找到对应的第三方模块,则移动到再上一层父目录中,进行加载,直到文件系统的根目录。

例如,假设在'C:\Users\itheima\project\foo.js'文件里调用了require('tools'),则Node.js会按以下顺序查找:

C:\Users\itheima\project\node_modules\tools

C:\Users\itheima\node_modules\tools

C:\Users\node_modules\tools

C:\node modules\tools

5.目录作为模块

当把目录作为模块标识符,传递给require()进行加载的时候,有三种加载方式:

1 在被加载的目录下查找一个叫做package.json的文件,并寻找main属性,作为require()加载的入口

2 如果目录里没有package.json文件,或者main入口不存在或无法解析,则Node.js将会试图加载目录下的index.js文件。

3 如果以上两步都失败了,则Node.js会在终端打印错误消息,报告模块的缺失:Error: Cannot find module'xxx'